

Wasla Platform

Technical Implementation Document

Graduation Project — Software Engineering

Backend Repository Analysis: wasla-backend

Stack: Next.js · Express · PostgreSQL · Prisma · FastAPI · JWT · Socket.IO

Table of Contents

1. User Login
2. User Registration
3. Create Post
4. View Home Feed
5. View Post Details
6. Apply to Post
7. Contact Post Owner
8. Create Service Contract
9. Record Work Session
- .0. Resolve Contract at Maximum End Date
- L1. View Recommended Posts
- .2. Appendix A — System Architecture Summary

Wasla Platform — Technical Implementation Document

Graduation Project — Software Engineering

System Stack: Next.js (Frontend) · Node.js / Express / TypeScript (Backend) · PostgreSQL / Prisma (Database) · FastAPI / Sentence Transformers / FAISS (AI Service) · JWT + Refresh Tokens (Authentication) · Socket.IO (Real-Time Communication)

Document Scope: This document describes the *actual* implementation of eleven core use cases as found in the Wasla backend repository (`wasla-backend`). All flows, modules, and code references correspond to production source code, not generic textbook patterns.

Table of Contents

1. [User Login](#1-user-login)
 2. [User Registration](#2-user-registration)
 3. [Create Post](#3-create-post)
 4. [View Home Feed](#4-view-home-feed)
 5. [View Post Details](#5-view-post-details)
 6. [Apply to Post](#6-apply-to-post)
 7. [Contact Post Owner](#7-contact-post-owner)
 8. [Create Service Contract](#8-create-service-contract)
 9. [Record Work Session](#9-record-work-session)
 10. [Resolve Contract at Maximum End Date](#10-resolve-contract-at-maximum-end-date)
 11. [View Recommended Posts](#11-view-recommended-posts)
-

1. User Login

1.1 Purpose

The User Login use case authenticates an existing Wasla user by verifying email and password credentials, establishes a server-side session record, and returns a short-lived JWT access token together with a long-lived refresh token. The access token authorizes subsequent REST API calls; the refresh token enables silent session renewal without re-entering credentials.

1.2 Actors

Actor	Role
-----	-----
Registered User	Submits login credentials via the Next.js frontend
Frontend Application	Sends <code>POST /auth/login</code> , stores <code>accessToken</code> in memory/state, relies on <code>httpOnly</code> cookie for refresh
Auth Module	Express routes, controller, and service under <code>src/modules/auth/</code>
PostgreSQL (via Prisma)	Stores <code>User</code> and <code>Session</code> entities
JWT Subsystem	Signs and verifies access/refresh tokens (<code>src/common/utils/jwt.ts</code>)

1.3 Preconditions

- The user account exists in the `users` table with a valid `password_hash`.
- The account has not been soft-deleted (`deleted_at IS NULL`).
- The client can reach the backend API and send JSON request bodies with CORS credentials enabled.

1.4 Trigger

The user submits the login form on the frontend, which issues:

```
POST /auth/login
Content-Type: application/json

{ "email": "...", "password": "..." }
```

1.5 Implementation Flow

1. **Rate limiting** — `loginLimiter(5, 60_000)` in `auth.routes.ts` restricts each IP to five login attempts per minute, returning HTTP 429 when exceeded.
2. **Request validation** — The `validate(loginSchema)` middleware parses the body with Zod (`emailSchema`, `passwordSchema`). Invalid payloads return HTTP 400 with structured field errors.
3. **Controller orchestration** — `loginController` extracts device metadata via `metaExtract(req)` (User-Agent parsing and client IP from `x-forwarded-for`), then delegates to `loginService`.
4. **Credential verification** — `loginService` loads the user by email, rejects missing or soft-deleted accounts with a generic "Invalid credentials" message (preventing user enumeration), and compares the plaintext password against `password_hash` using `bcrypt.compare`.
5. **Session creation** — On success, `createSession(userId, meta)` generates JWT access (15 min) and refresh (7 days) tokens, persists a `Session` row with UUID primary key, refresh token, expiry, device info, and IP address.
6. **Response assembly** — The controller sets the refresh token as an `httpOnly` cookie via `setRefreshTokenCookie` and returns JSON `{ accessToken, user: { id, email, username } }`.

Architecturally, login follows a **thin controller / fat service** pattern with middleware-based cross-cutting concerns (validation, rate limiting).

1.6 Business Rules

- Email and password are mandatory at the service layer even after Zod validation.
- Invalid email/password combinations always yield the same error message.
- Email verification (`is_verified`) exists on the `User` model but the login check is currently commented out in `login.service.ts`.
- Password must satisfy complexity rules enforced at registration/login schema level (minimum 8 characters, uppercase, lowercase, digit, special character).

1.7 Database Operations

Operation	Entity	Prisma Call
-----	-----	-----
Read	User	<code>prisma.user.findUnique({ where: { email } })</code>
Create	Session	<code>prisma.session.create({ data: { id, user_id, refresh_token, expires_at, device_info, ip_address } })</code>

No updates or deletes occur during login.

1.8 Security Considerations

- **Password storage:** bcrypt with cost factor 10 (applied at registration).
- **Token separation:** Access token in JSON body; refresh token in httpOnly cookie (`secure` in production, `sameSite: "none"` in production for cross-origin SPA support).
- **Rate limiting:** Mitigates brute-force attacks on `/auth/login`.
- **Generic error messages:** Prevents distinguishing between unknown email and wrong password.
- **Soft-delete enforcement:** Deleted users cannot authenticate.
- **Trust proxy:** `app.set("trust proxy", 1)` ensures correct client IP behind reverse proxies for session metadata.

1.9 Error Handling

Condition	HTTP Status	Response Shape
-----	-----	-----
Zod validation failure	400	<code>{ status: "fail", errors: [{ path, message }] }</code>
Invalid credentials	400	<code>{ status: "fail", message: "Invalid credentials" }</code>
Session creation failure	400	<code>{ status: "fail", message: "Session creation failed" }</code>
Rate limit exceeded	429	<code>{ status: "fail", message: "Too many login attempts..." }</code>
Malformed JSON body	400	<code>{ message: "Invalid JSON request body" }</code>

Controllers use `sendError` from `httpError.ts` for consistent failure responses.

1.10 Code References

Snippet 1 — Route definition with middleware chain

File: `src/modules/auth/auth.routes.ts`

```
router.post(
  "/login",
  loginLimiter(5, 60 * 1000),
  validate(loginschema),
  loginController,
);
```

This snippet defines the public login endpoint mounted at `/auth/login`. Rate limiting runs first, then Zod validation, then the controller. No authentication middleware is applied because the user is not yet authenticated.

Snippet 2 — Login controller and token delivery

File: `src/modules/auth/auth.controller.ts`

```
export const loginController = async (req: Request, res: Response) => {
  try {
    const meta = await metaExtract(req);
    const result = await loginService(req.body, meta);
    setRefreshTokenCookie(res, result.refreshToken);

    return res.json({
      accessToken: result.accessToken,
      user: result.user,
    });
  } catch (err: unknown) {
    return sendError(res, 400, getErrorMessage(err, "Login failed"));
  }
};
```

The controller separates transport concerns (cookie setting, JSON response) from business logic. The refresh token never appears in the JSON body, reducing XSS exposure.

Snippet 3 — Credential verification and session bootstrap

File: `src/modules/auth/login.service.ts`

```
const user = await prisma.user.findUnique({
  where: { email },
});

if (!user || user.deleted_at) {
  throw new Error(`Invalid credentials`);
}

const isValid = await bcrypt.compare(password, user.password_hash);
if (!isValid) {
  throw new Error(`Invalid credentials`);
}

const { refreshToken, accessToken } = await createSession(user.id, meta);
```

This is the core authentication gate. Both missing users and incorrect passwords produce identical errors. Successful authentication delegates session persistence to `createSession`.

Snippet 4 — Session and JWT creation

File: `src/modules/auth/register/create_session.ts`

```
const refreshToken = RefreshAccessToken(userId.toString());
const accessToken = signAccessToken(userId.toString());
await prisma.session.create({
  data: {
    id: uuidv4(),
    user_id: userId,
    refresh_token: refreshToken,
    expires_at: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000),
    device_info: meta.deviceInfo,
    ip_address: meta.ip,
  },
});
return { refreshToken, accessToken };
```

Each login creates a new session row, enabling multi-device support and refresh token rotation on subsequent `/auth/refresh` calls.

2. User Registration

2.1 Purpose

User Registration onboards a new participant into the Wasla time-credit exchange platform. It creates a user profile, associates skill tags for the AI recommender, grants an initial welcome bonus of five time credits, establishes an authenticated session, and synchronizes the new user vector to the FastAPI recommender service.

2.2 Actors

Actor	Role
-----	-----
Prospective User	Completes the registration form on the frontend
Auth Module	<code>RegisterControler</code> , <code>RegisterService</code> , validation schemas
Skills Module	<code>syncUserSkillsByType</code> links offered/required skills
Wallet Subsystem	Creates <code>WELCOME_BONUS</code> transaction
AI Recommender Service	Receives user profile sync via <code>POST /sync/user</code>
PostgreSQL	Persists <code>User</code> , <code>UserSkill</code> , <code>Skill</code> , <code>Transaction</code> , <code>Session</code>

2.3 Preconditions

- Email and username are not already registered.
- The client submits all required registration fields including at least one offered skill and one required skill.
- The recommender service may be offline; registration succeeds regardless (sync is fire-and-forget).

2.4 Trigger

```
POST /auth/register
Content-Type: application/json

{
  "full_name", "username", "email", "password",
  "bio?", "profile_image?", "location?",
  "offeredSkills": ["..."],
  "requiredSkills": ["..."]
}
```

2.5 Implementation Flow

1. **Validation** — `validate(registerSchema)` enforces name/username/email/password rules and skill array constraints (1–10 unique items, minimum 2 characters each).
2. **Uniqueness checks** — `RegisterService` queries `User` by email and username independently before any write.
3. **Password hashing** — `bcrypt.hash(password, 10)` produces `password_hash`.
4. **Atomic transaction** — A Prisma `$transaction` (15 s timeout) creates the user, syncs skills of type `OFFER` and `REQUEST`, and records a welcome bonus transaction crediting five time credits.
5. **Session creation** — `createSession` issues JWT pair and persists session metadata (same path as login).
6. **Recommender sync** — `syncUser(user.id)` is invoked asynchronously to index the user in the FAISS-based recommender.
7. **Response** — Controller sets refresh cookie and returns `{ accessToken, user: { id, email, username } }`.

Design decision: registration immediately authenticates the user (no separate login step), reducing onboarding friction.

2.6 Business Rules

- Username must contain at least three English letters; Arabic letters allowed in full name only.
- Bio, if provided, must be 50–200 characters.
- Profile image must be a valid URL when supplied.
- Duplicate emails or usernames are rejected before the transaction begins.
- Welcome bonus: `amount: 5`, `transaction_type: "WELCOME_BONUS"`, `sender_id: null`.
- Skills are deduplicated at schema level and upserted into the `skills` / `user_skills` tables.

2.7 Database Operations

Operation	Entity	Details
-----	-----	-----
Read	User	Uniqueness checks for email and username
Create	User	Profile fields and hashed password
Create/Upsert	Skill , UserSkill	Via <code>syncUserSkillsByType</code> inside transaction
Create	Transaction	Welcome bonus credit
Create	Session	Via <code>createSession</code> after transaction

2.8 Security Considerations

- Strong password policy enforced by Zod regex rules.
- Password never stored in plaintext; only bcrypt hash persisted.
- Registration endpoint is **not** rate-limited (unlike login) — a potential hardening target.
- Immediate session issuance follows the same secure cookie pattern as login.
- Input sanitization via Zod `trim()` on string fields.

2.9 Error Handling

Condition	HTTP Status	Message
-----	-----	-----
Zod validation failure	400	Structured field errors
Duplicate email	400	"Email already used"
Duplicate username	400	"Username is already taken"
Transaction timeout/failure	400	"Register failed"
Missing request body	400	"Request body is required"

2.10 Code References

Snippet 1 — Registration validation schema

File: `src/modules/auth/auth.schema.ts`

```
export const registerSchema = z.object({
  full_name: z.string().min(3).max(100).regex(/^[a-zA-Z\u0600-\u06FF\s]+$/),
  username: z.string().trim().min(3).max(50).regex(/^(?=[a-zA-Z]{3,})[a-zA-Z0-9\d\W_]+$/),
  email: emailSchema,
  password: passwordSchema,
  bio: z.string().min(50).max(200).optional().or(z.literal("")),
  profile_image: z.string().url().optional(),
  location: z.string().min(3).optional(),
  offeredSkills: userSkillsArraySchema,
  requiredSkills: userSkillsArraySchema,
});
```

This schema is the single source of truth for registration input validation, shared between runtime middleware and OpenAPI documentation.

Snippet 2 — Atomic user creation with skills and welcome bonus

File: `src/modules/auth/register/register.service.ts`

```
const user = await prisma.$transaction(async (tx) => {
  const user = await tx.user.create({
    data: { full_name, username, email, password_hash: hashPassword, ... },
  });

  await syncUserSkillsByType(tx, user.id, offeredSkills, "OFFER");
  await syncUserSkillsByType(tx, user.id, requiredSkills, "REQUEST");

  await tx.transaction.create({
    data: {
      receiver_id: user.id,
      sender_id: null,
      amount: 5,
      transaction_type: "WELCOME_BONUS",
    },
  });

  return user;
}, { timeout: 15000 });
```

The transaction ensures that a user is never created without skills and initial credits, maintaining data consistency.

Snippet 3 — Post-registration session and AI sync

File: `src/modules/auth/register/register.service.ts`

```
const { refreshToken, accessToken } = await createSession(user.id, meta);
syncUser(user.id);
return { id: user.id, email: user.email, username: user.username, refreshToken,
accessToken };
```

`syncUser` calls the FastAPI recommender (`POST /sync/user`) without blocking the HTTP response, following an eventual-consistency model for the AI index.

Snippet 4 — Registration controller response

File: `src/modules/auth/register/register.controller.ts`

```
const { id, email, username, refreshToken, accessToken } =
  await RegisterService(data, meta);

setRefreshTokenCookie(res, refreshToken);

return res.json({
  accessToken,
  user: { id, email, username },
});
```

3. Create Post

3.1 Purpose

Create Post allows an authenticated user to publish a service offer or request on the Wasla marketplace. Each post defines title, description, category (`OFFER` or `REQUEST`), service mode (`ONLINE` or `OFFLINE`), assigned time credits, optional geographic fields, and publication status. Upon creation, the post is indexed in the AI recommender for personalized feed generation.

3.2 Actors

Actor	Role
-----	-----
Authenticated User	Post author
Posts Module	Routes, controller, service, schemas under <code>src/modules/posts/</code>
AI Recommender Service	Receives post embedding via <code>syncPost</code> → <code>POST /sync/post</code>
PostgreSQL	<code>Post</code> entity with related <code>User</code>

3.3 Preconditions

- Valid JWT access token in `Authorization: Bearer` header.
- Request body passes `createPostSchema` validation.
- For `OFFLINE` service mode, both `city` and `area` must be provided.

3.4 Trigger

```
POST /posts
Authorization: Bearer <accessToken>
Content-Type: application/json
```

3.5 Implementation Flow

1. **Authentication** — `authMiddleware` verifies JWT and attaches `req.user.userId`.
2. **Validation** — `validate(createPostSchema)` enforces field lengths, enums, and offline location requirement.

3. 3. **Controller** — `createPostController` extracts numeric user ID via `getUserId(req)` and passes validated body to the service.
4. 4. **Persistence** — `createPostService` inserts a `Post` row with snake_case column mapping (`service_mode`, `assigned_time_credits`, etc.).
5. 5. **Response mapping** — `toPostResponse` converts database record to camelCase API shape including nested author `user` object.
6. 6. **AI indexing** — `syncPost(post.id)` fire-and-forgets a recommender sync call.
7. 7. **Response** — HTTP 201 with `{ post }`.

Route ordering in `posts.routes.ts` places `GET /` (public feed) before `authMiddleware`, while `POST /` requires authentication.

3.6 Business Rules

- Title: 5–200 characters; description: 10–5000 characters.
- `assignedTimeCredits`: positive integer, max 100,000.
- Default status is `PUBLISHED` (Prisma schema default) unless client specifies `DRAFT` or `ARCHIVED`.
- Offline posts require geographic fields; online posts may omit them.
- Only the authenticated user's ID is stored as `user_id`; no impersonation path exists.

3.7 Database Operations

Operation	Entity	Prisma Call
-----	-----	-----
Create	<code>Post</code>	<code>prisma.post.create({ data: { user_id, title, description, category, service_mode, assigned_time_credits, status, city, area }, select: postSelect })</code>

`postSelect` joins author profile fields (`username`, `full_name`, `profile_image`).

3.8 Security Considerations

- JWT authentication required; unauthenticated requests receive 401.
- User ID derived from token, not from request body — prevents author spoofing.
- Input validation prevents oversized payloads and invalid enum injection.
- Post content is stored as provided; no server-side HTML sanitization layer is present.

3.9 Error Handling

Condition	HTTP Status	Message
-----	-----	-----
Missing/invalid token	401	"Unauthorized"
Zod validation failure	400	Structured field errors
Database error	400	"Create post failed"

3.10 Code References

Snippet 1 — Route and middleware

File: `src/modules/posts/posts.routes.ts`

```
router.get("/", listPublishedPostsController)
router.use(authMiddleware)
router.post("/", validate(createPostSchema), createPostController)
```

Public read access for the home feed is explicitly separated from authenticated write operations.

Snippet 2 — Create post service with recommender sync

File: `src/modules/posts/posts.service.ts`

```
export const createPostService = async (data: CreatePostInput, userId: number) => {
  const post = await prisma.post.create({
    data: {
      user_id: userId,
      title: data.title,
      description: data.description,
      category: data.category,
      service_mode: data.serviceMode,
      assigned_time_credits: data.assignedTimeCredits,
      status: data.status,
      city: data.city,
      area: data.area,
    },
    select: postSelect,
  })
  syncPost(post.id)
  return toPostResponse(post)
}
```

Snippet 3 — Offline location validation

File: `src/modules/posts/posts.schema.ts`


```
export const createPostSchema = baseCreatePostSchema.refine(
  (data) => {
    if (data.serviceMode === "OFFLINE") {
      return !!data.city && !!data.area;
    }
    return true;
  },
  { message: "City and Area are required for offline services", path: ["city"] }
);
```

Snippet 4 — Controller

File: src/modules/posts/posts.controller.ts

```
export const createPostController = async (req: Request, res: Response) => {
  const userId = getUserId(req)
  const data: CreatePostInput = req.body;
  const post = await createPostService(data, userId);
  return res.status(201).json({ post });
};
```

4. View Home Feed

4.1 Purpose

View Home Feed retrieves a paginated, chronologically ordered list of all published posts. This is the platform's public discovery feed, independent of user authentication or AI personalization. It serves as the default content stream and as the fallback when the recommender service is unavailable.

4.2 Actors

Actor	Role
-----	-----
Visitor / Authenticated User	Browses posts on the home page
Posts Module	<code>listPublishedPostsController</code> , <code>listPublishedPostsService</code>
Pagination Utilities	<code>buildPostCursorFilter</code> , <code>paginateById</code>
PostgreSQL	<code>Post</code> with <code>status = PUBLISHED</code>

4.3 Preconditions

- No authentication required.
- Optional query parameters `cursor` and `limit` must conform to `listPostsQuerySchema` when provided.

4.4 Trigger

```
GET /posts?cursor=<postId>&limit=20
```

4.5 Implementation Flow

1. **Public route** — `GET /` is registered before `authMiddleware` in `posts.routes.ts` , making the endpoint accessible without a token.
2. **Query parsing** — Controller parses `req.query` with `listPostsQuerySchema` (default `limit: 20` , max 50).

- 3. 3. **Cursor resolution** — `buildPostCursorFilter(cursor)` loads the cursor post's `created_at` and `id`, then builds a keyset filter for `(created_at DESC, id DESC)` pagination.
- 4. 4. **Database query** — `prisma.post.findMany` fetches `limit + 1` published posts with `postSelect` (includes author profile).
- 5. 5. **Pagination slicing** — `paginateById` returns `limit` items and sets `nextCursor` to the last item's ID if more results exist.
- 6. 6. **Response** — `{ posts: [...], nextCursor: number | null }`.

Architectural pattern: **cursor-based keyset pagination** avoids OFFSET performance degradation on large datasets.

4.6 Business Rules

- Only posts with `status: "PUBLISHED"` appear in the feed.
- Ordering is strictly by `created_at DESC, id DESC`.
- Invalid or unknown cursor values are silently ignored (returns first page).
- Maximum page size is 50 posts.

4.7 Database Operations

Operation	Entity	Details
-----	-----	-----
Read	Post	Cursor lookup + paginated <code>findMany</code> with author join
Read	User	Via <code>postSelect.user</code> nested select

No writes occur.

4.8 Security Considerations

- Public endpoint exposes only published posts.
- Draft and archived posts are excluded at the query level.
- No user-specific data beyond public author profiles is returned.

4.9 Error Handling

Condition	HTTP Status	Message
-----	-----	-----
Invalid query parameters	400	"Invalid request data"
Database failure	400	"Fetch posts failed"

4.10 Code References

Snippet 1 — Public route registration

File: `src/modules/posts/posts.routes.ts`

```
router.get("/", listPublishedPostsController)
router.use(authMiddleware)
```

Snippet 2 — Feed service with cursor pagination

File: `src/modules/posts/posts.service.ts`

```
export const listPublishedPostsService = async (query: ListPostsQuery) => {
  const limit = query.limit ?? 20
  const cursorFilter = await buildPostCursorFilter(query.cursor)

  const posts = await prisma.post.findMany({
    where: { status: "PUBLISHED", ...cursorFilter },
    orderBy: [{ created_at: "desc" }, { id: "desc" }],
    take: limit + 1,
    select: postSelect,
  })

  const { items, nextCursor } = paginateById(posts, limit)
  return { posts: items.map(toPostResponse), nextCursor }
}
```

Snippet 3 — Keyset cursor filter

File: `src/modules/posts/posts.pagination.ts`

```
return {
  OR: [
    { created_at: { lt: cursorPost.created_at } },
    { AND: [{ created_at: cursorPost.created_at }, { id: { lt: cursorPost.id } } ] },
  ],
};
```

Snippet 4 — Pagination helper

File: `src/modules/posts/posts.pagination.ts`

```
export const paginateById = <T extends { id: number }>(items: T[], limit: number) => {  
  const hasMore = items.length > limit;  
  const page = hasMore ? items.slice(0, limit) : items;  
  return { items: page, nextCursor: hasMore ? (page[page.length - 1]?.id ?? null) : null  
};  
};
```

5. View Post Details

5.1 Purpose

View Post Details retrieves the full representation of a single post, including author profile metadata. It enforces visibility rules so that non-published posts are accessible only to their owner, while published posts are visible to any authenticated user.

5.2 Actors

Actor	Role
-----	-----
Authenticated User	Views post detail page
Posts Module	<code>getPostByIdController</code> , <code>getPostByIdService</code>
PostgreSQL	<code>Post</code> , <code>User</code>

5.3 Preconditions

- Valid JWT access token.
- `postId` path parameter is a positive integer (coerced by `postIdParamSchema`).

5.4 Trigger

```
GET /posts/:postId
Authorization: Bearer <accessToken>
```

5.5 Implementation Flow

1. **Authentication** — `authMiddleware` validates JWT.
2. **Parameter validation** — `validate(postIdParamSchema, "params")` coerces `postId` to a positive integer.
3. **Lookup** — `getPostByIdService` performs `prisma.post.findUnique` with extended select including `user_id` and `status` .
4. **Access control** — If post not found → error. If status is not `PUBLISHED` and caller is not the owner → `"You cannot view this post"` .

5. 5. **Response** — `{ post: toPostResponse(post) }`.

Unlike the home feed, post details require authentication even for published posts.

5.6 Business Rules

- Published posts: visible to all authenticated users.
- Draft/Archived posts: visible only to `post.user_id === callerId`.
- Missing posts return 404.

5.7 Database Operations

Operation	Entity	Details
-----	-----	-----
Read	Post	<code>findUnique</code> with <code>postSelect</code> + status + user_id
Read	User	Nested author select

5.8 Security Considerations

- Authentication required for all detail views.
- Owner-only access for non-published content prevents draft leakage.
- Post ID validated as positive integer, preventing injection via path params.

5.9 Error Handling

Condition	HTTP Status	Message
-----	-----	-----
Unauthorized	401	"Unauthorized"
Post not found	404	"Post not found"
Access denied	404	"You cannot view this post"

Note: Access denied returns 404 rather than 403, obscuring existence of private drafts.

5.10 Code References

Snippet 1 — Route definition

File: `src/modules/posts/posts.routes.ts`

```
router.get("/:postId", validate(postIdParamSchema, "params"), getPostByIdController)
```

Snippet 2 — Access control logic

File: src/modules/posts/posts.service.ts

```
export const getPostByIdService = async (postId: number, userId: number) => {
  const post = await prisma.post.findUnique({
    where: { id: postId },
    select: { ...postSelect, user_id: true, status: true },
  })
  if (!post) {
    throw new Error("Post not found")
  }
  if (post.status !== "PUBLISHED" && post.user_id !== userId) {
    throw new Error("You cannot view this post")
  }
  return toPostResponse(post)
}
```

Snippet 3 — Controller

File: src/modules/posts/posts.controller.ts

```
export const getPostByIdController = async (req: Request, res: Response) => {
  const userId = getUserId(req)
  const postId = req.params.postId as unknown as number;
  const post = await getPostByIdService(postId, userId)
  return res.json({ post })
};
```

Snippet 4 — Post ID param schema

File: src/modules/posts/posts.schema.ts

```
export const postIdParamSchema = z.object({
  postId: z.coerce.number().int().positive(),
});
```


6. Apply to Post

6.1 Purpose

Apply to Post initiates a service exchange request against a published post. The requester proposes a duration in time credits and a maximum contract end date. The system validates credit availability, creates a pending `ServiceExchange` record (the platform's application/contract entity), notifies the provider, and records an interaction signal for the AI recommender.

6.2 Actors

Actor	Role
-----	-----
Requester (Consumer)	User applying for a service
Provider	Post owner / service provider (receives notification)
Exchanges Module	<code>createExchangeController</code> , <code>requestExchange</code>
Notifications Module	<code>createContractNotification</code>
AI Recommender	Receives <code>apply</code> interaction via <code>syncInteraction</code>
PostgreSQL	<code>ServiceExchange</code> , <code>Post</code> , <code>User</code>

6.3 Preconditions

- Requester is authenticated.
- Requester is not the provider (`providerId !== requesterId`).
- Referenced post and provider exist.
- Requester has `available_balance >= duration` (pre-check only; credits are not deducted until acceptance).

6.4 Trigger

```
POST /exchanges/request
Authorization: Bearer <accessToken>

{
  "postId": number,
  "providerId": number,
  "duration": number,
  "maximumEndDate": ISO8601 date (must be in future)
}
```

6.5 Implementation Flow

1. **Authentication and validation** — All exchange routes use `authMiddleware` ; body validated by `createExchangeSchema` .
2. **Self-request guard** — Rejected if requester equals provider.
3. **Entity validation** — Post and provider looked up; requester's `available_balance` checked.
4. **Contract creation** — `ServiceExchange` created with `status: PENDING` , `escrow_status: NONE` , linking `post_id` , `provider_id` , `consumer_id` , `time_credits` , and `maximum_end_date` .
5. **Recommender signal** — `syncInteraction({ userId, postId, action: "apply" })` sent to FastAPI.
6. **Notification** — Provider receives `EXCHANGE_REQUESTED` in-app notification.
7. **Response** — Exchange object mapped via `toExchangeResponse` (camelCase API shape).

There is no separate `Application` table; the pending exchange *is* the application.

6.6 Business Rules

- Duration: positive integer, max 100,000.
- `maximumEndDate` must be strictly in the future.
- No credit deduction at request time — only a balance pre-check.
- One request creates one `PENDING` exchange; duplicate handling is left to business logic upstream.
- Credits remain in requester's available balance until provider accepts.

6.7 Database Operations

Operation	Entity	Details
-----	-----	-----
Read	Post	Existence check
Read	User	Provider existence; requester balance
Create	ServiceExchange	Pending contract record
Create	Notification	Provider alert (async, non-blocking)

6.8 Security Considerations

- JWT authentication required.
- Requester ID from token, not body.
- Self-service requests blocked.
- Balance check prevents obviously unfulfillable requests (final atomic hold occurs at acceptance).

6.9 Error Handling

Condition	HTTP Status	Message
-----	-----	-----
Self-request	400	"You cannot request a service from yourself"
Post not found	404	"Post not found"
Insufficient credits	400	"Insufficient time credits"
Validation failure	400	Zod field errors

Uses `ExchangeError` with explicit status codes, caught by controller.

6.10 Code References

Snippet 1 — Route

File: `src/modules/exchanges/exchanges.routes.ts`

```
router.post("/request", validate(createExchangeSchema), createExchangeController);
```

Snippet 2 — Request validation schema

File: `src/modules/exchanges/exchanges.schema.ts`

```
export const createExchangeSchema = z.object({
  postId: z.coerce.number().int().positive(),
  providerId: z.coerce.number().int().positive(),
  duration: z.coerce.number().int().positive().max(100000),
  maximumEndDate: z.coerce.date().refine(val => val > new Date(), {
    message: "Maximum end date must be in the future"
  }),
});
```

Snippet 3 — Core apply logic

File: src/modules/exchanges/exchanges.service.ts

```
export const requestExchange = async (requesterId: number, data: CreateExchangeInput) =>
{
  if (data.providerId === requesterId) {
    throw new ExchangeError("You cannot request a service from yourself", 400);
  }
  // ... validation ...
  if (requester.available_balance < data.duration) {
    throw new ExchangeError("Insufficient time credits", 400);
  }

  const exchange = await prisma.serviceExchange.create({
    data: {
      post_id: data.postId,
      provider_id: data.providerId,
      consumer_id: requesterId,
      time_credits: data.duration,
      maximum_end_date: data.maximumEndDate,
      status: "PENDING",
      escrow_status: "NONE",
    },
  });

  syncInteraction({ userId: requesterId, postId: data.postId, action: "apply" });
  // ... notification ...
};
```

Snippet 4 — Prisma ServiceExchange model

File: prisma/schema.prisma

```
model ServiceExchange {  
    id                Int                @id @default(autoincrement())  
    post_id           Int?  
    provider_id       Int  
    consumer_id       Int  
    time_credits      Int  
    completed_hours   Int                @default(0)  
    status             ServiceExchangeStatus @default(PENDING)  
    escrow_status      EscrowStatus        @default(NONE)  
    maximum_end_date  DateTime  
    work_sessions     WorkSession[]  
    @@map("service_exchanges")  
}
```

7. Contact Post Owner

7.1 Purpose

Contact Post Owner enables direct messaging between a user and a post's author. The system creates or reuses a post-linked conversation, then delivers messages in real time via Socket.IO with persistent storage in PostgreSQL. This supports pre-contract negotiation and coordination.

7.2 Actors

Actor	Role
-----	-----
Caller	User initiating contact (typically an applicant)
Post Owner	Recipient when caller is not the owner
Chat Module	<code>chat.service.ts</code> , <code>message.service.ts</code> , controllers
Socket.IO Server	<code>src/realtime/socket.ts</code> , emit helpers
Notifications Module	Message notifications and optional email
PostgreSQL	<code>Conversation</code> , <code>ConversationParticipant</code> , <code>Message</code>

7.3 Preconditions

- Caller is authenticated.
- Referenced post exists.
- If caller is the post owner, `recipientId` must be explicitly provided.
- Caller and recipient must be distinct users.

7.4 Trigger

Step 1 — Create or get conversation:

```
POST /conversations
{ "postId": 123 }           // recipient auto-resolved to post owner
```

Step 2 — Send message:

```
POST /conversations/:conversationId/messages
{ "body": "...", "clientId": "<uuid>" }
```

7.5 Implementation Flow

Conversation creation:

1. Load post; determine recipient (auto = owner if caller ≠ owner).
2. Validate recipient exists and is not soft-deleted.
3. Search for existing conversation with same post and both participants.
4. If found, return existing conversation (`isNew: false`).
5. Otherwise, create `Conversation` with `postId` and two `ConversationParticipant` rows.

Message sending:

1. Verify caller is a conversation participant (`assertConversationParticipant`).
2. Persist `Message` with optional `clientId` for idempotency (unique constraint).
3. Fire-and-forget side effects: emit `chat:message:sent` to sender, `chat:message:new` to conversation room, create notification, emit `chat:notification:new`.
4. Rate limit: 30 messages per minute per user on the send endpoint.

Real-time delivery uses Socket.IO rooms keyed by `conversationId`. Clients join via `chat:join` event after connecting with JWT in `socket.handshake.auth.token`.

7.6 Business Rules

- Post owner contacting someone must specify `recipientId`.
- Non-owner contacting a post auto-targets the owner.
- Self-messaging is prohibited.
- One post-linked conversation per participant pair is reused (deduplication).
- Messages require non-empty body validated by `sendMessageSchema`.

7.7 Database Operations

Operation	Entity	Details
-----	-----	-----
Read	Post	Resolve owner
Read	User	Recipient validation
Read/Create	Conversation	Dedup or create with participants
Create	Message	Persist message body and metadata
Create	Notification	Async message notification

7.8 Security Considerations

- JWT on REST endpoints; JWT on Socket.IO handshake.
- Participant guard prevents sending to conversations the user does not belong to.
- Message rate limiting mitigates spam.
- `clientId` unique constraint prevents duplicate message insertion on retry.

7.9 Error Handling

Condition	HTTP Status	Message
-----	-----	-----
Post not found	404	"Post not found"
Missing recipientId (owner case)	400	"You must provide recipientId"
Not a participant	403	Chat guard error
Duplicate clientId	200	Returns existing message (idempotent)

7.10 Code References

Snippet 1 — Conversation creation with owner resolution

File: `src/modules/chat/chat.service.ts`


```
export const createOrGetConversation = async (callerId: number, input:
CreateConversationInput) => {
  const post = await prisma.post.findUnique({ where: { id: input.postId } });
  if (!post) throw new ChatError("Post not found", 404);

  let targetRecipientId: number;
  if (callerId === post.user_id) {
    if (!input.recipientId) throw new ChatError("You must provide recipientId", 400);
    targetRecipientId = input.recipientId;
  } else {
    targetRecipientId = post.user_id;
  }
  // ... dedupe or create ...
};
```

Snippet 2 — Socket.IO message broadcast

File: src/modules/chat/message.service.ts

```
fireAndForget("chat:message-side-effects", async () => {
  emitToUser(callerId, "chat:message:sent", response);
  emitToConversation(conversationId, "chat:message:new", response);
  const notification = await createMessageNotification({ ... });
  emitToConversation(conversationId, "chat:notification:new", notification);
});
```

Snippet 3 — Chat routes

File: src/modules/chat/chat.routes.ts

```
router.post("/", validate(createConversationSchema), createConversationController);
router.post("/:conversationId/messages", messageRateLimit(30, 60 * 1000),
  validate(sendMessageSchema), sendMessageController);
```

Snippet 4 — Conversation schema

File: src/modules/chat/chat.schema.ts

```
export const createConversationSchema = z.object({
  postId: z.coerce.number().int().positive(),
  recipientId: z.coerce.number().int().positive().optional(),
});
```

8. Create Service Contract

8.1 Purpose

In Wasla, a service contract is materialized as a `ServiceExchange` entity. Contract creation occurs in two phases: (1) **Request Phase** — the consumer applies via `POST /exchanges/request`, creating a `PENDING` contract; (2) **Activation Phase** — the provider accepts via `PUT /exchanges/:id/accept`, atomically holding time credits in escrow and transitioning the contract to `IN_PROGRESS`. This use case documents the full contract lifecycle from creation through activation.

8.2 Actors

Actor	Role
-----	-----
Consumer (Requester)	Initiates contract request
Provider	Accepts or rejects the contract
Exchanges Module	Request and acceptance services
Wallet/Escrow Logic	Credit hold in <code>User.available_balance</code> → <code>User.escrow_balance</code>
Notifications Module	Contract state change alerts
PostgreSQL	<code>ServiceExchange</code> , <code>User</code> , <code>Transaction</code>

8.3 Preconditions

- **Request phase:** Same as Apply to Post (Section 6).
- **Activation phase:** Contract exists in `PENDING` status; caller is the provider; requester still has sufficient available credits at acceptance time.

8.4 Trigger

Phase 1 — Request (contract creation):

```
POST /exchanges/request
```

Phase 2 — Activation (contract becomes binding):

```
PUT /exchanges/:id/accept
Authorization: Bearer <providerToken>
```

8.5 Implementation Flow

Phase 1 — PENDING contract (see Section 6): Creates exchange with `escrow_status: NONE`. No funds moved.

Phase 2 — Acceptance (`acceptExchange`):

1. Serializable transaction begins (`runSerializable`).
2. Load exchange; verify caller is provider and status is `PENDING`.
3. **Atomic escrow hold** — `updateMany` on requester's user row with condition `available_balance >= time_credits`, decrementing available and incrementing escrow atomically. If count is 0, requester no longer has funds.
4. **Status transition** — Exchange updated to `status: IN_PROGRESS`, `escrow_status: HELD`, `accepted_at: now()`.
5. Optimistic concurrency via `updateMany` with status condition prevents double-acceptance.
6. Provider and consumer notified.

Design decision: Escrow hold at acceptance (not at request) protects requesters from premature credit lock-up and allows providers to reject without financial side effects.

8.6 Business Rules

- Only provider may accept.
- Acceptance requires `PENDING` status.
- Credit hold is race-safe via conditional `updateMany`.
- Contract fields: `time_credits` (agreed duration), `maximum_end_date` (deadline), `completed_hours` (tracks confirmed work, default 0).
- Rejection sets status to `REJECTED` without financial movement.

8.7 Database Operations

Phase	Operation	Entity
-----	-----	-----
Request	Create	<code>ServiceExchange</code> (PENDING)
Accept	Update	<code>User</code> (balance transfer to escrow)
Accept	Update	<code>ServiceExchange</code> (IN_PROGRESS, HELD)
Accept	Create	<code>Transaction</code> (escrow hold ledger entry)

8.8 Security Considerations

- Provider-only acceptance enforced by comparing `exchange.provider_id === providerId`.
- Serializable isolation level prevents concurrent acceptance race conditions.
- Double-spend prevented by conditional balance update.

8.9 Error Handling

Condition	HTTP Status	Message
-----	-----	-----
Not provider	403	"Only the provider can accept this exchange"
Not pending	400	"Exchange is not pending"
Insufficient credits at accept	400	"Requester no longer has enough time credits"
Concurrent accept	409	"Exchange is no longer pending"

8.10 Code References

Snippet 1 — Atomic escrow hold on acceptance

File: `src/modules/exchanges/exchanges.service.ts`

```
const held = await tx.user.updateMany({
  where: {
    id: exchange.consumer_id,
    available_balance: { gte: exchange.time_credits },
  },
  data: {
    available_balance: { decrement: exchange.time_credits },
    escrow_balance: { increment: exchange.time_credits },
  },
});
if (held.count === 0) {
  throw new ExchangeError("Requester no longer has enough time credits", 400);
}

await tx.serviceExchange.updateMany({
  where: { id, status: "PENDING" },
  data: { status: "IN_PROGRESS", escrow_status: "HELD", accepted_at: new Date() },
});
```

Snippet 2 — Accept route

File: src/modules/exchanges/exchanges.routes.ts

```
router.put("/:id/accept", acceptExchangeController);
```

Snippet 3 — Escrow status enum

File: prisma/schema.prisma

```
enum EscrowStatus {
  NONE
  HELD
  RELEASED
  REFUNDED
}
```

Snippet 4 — Contract status enum

File: prisma/schema.prisma

```
enum ServiceExchangeStatus {  
    PENDING  
    ACCEPTED  
    IN_PROGRESS  
    WAITING_CONFIRMATION  
    COMPLETED  
    CANCELED  
    REJECTED  
    DISPUTED  
}
```

9. Record Work Session

9.1 Purpose

Record Work Session allows the service provider to log completed work hours against an active contract. Each session enters a `PENDING_CONFIRMATION` state awaiting consumer approval. Upon confirmation, hours accumulate toward `completed_hours` on the contract, and the contract auto-completes when confirmed hours equal agreed `time_credits`.

9.2 Actors

Actor	Role
-----	-----
Provider	Records session hours
Consumer (Requester)	Confirms or rejects sessions
Exchanges Module	<code>recordWorkSession</code> , <code>confirmWorkSession</code>
Notifications Module	<code>SESSION_RECORDED</code> alerts
PostgreSQL	<code>WorkSession</code> , <code>ServiceExchange</code>

9.3 Preconditions

- Contract status is `IN_PROGRESS` or `WAITING_CONFIRMATION`.
- Caller is the provider.
- Total hours (completed + pending + new) does not exceed `time_credits`.

9.4 Trigger

```
POST /exchanges/:id/sessions
Authorization: Bearer <providerToken>

{ "hours": number, "notes": "optional string" }
```

9.5 Implementation Flow

1. **Validation** — `createSessionSchema` enforces positive integer hours (max 100,000).

2. 2. **Serializable transaction** — Load contract; verify provider identity and active status.
3. 3. **Hours cap check** — Sum `completed_hours` + pending session hours + new hours; reject if exceeds `time_credits`.
4. 4. **Session numbering** — Auto-increment `session_number` per contract (unique constraint on `[contract_id, session_number]`).
5. 5. **Create** — `WorkSession` with `status: PENDING_CONFIRMATION`.
6. 6. **Notify** — Consumer receives `SESSION_RECORDED` notification.
7. 7. **Confirmation flow** (separate endpoint) — Consumer calls `PUT /exchanges/:id/sessions/:sessionId/confirm`; hours added to `completed_hours`; if `completed_hours == time_credits`, contract auto-completes with escrow release.

9.6 Business Rules

- Only provider may record sessions.
- Hours must be positive integers.
- Cumulative hours (confirmed + pending + new) $\leq$ agreed credits.
- Sessions require explicit consumer confirmation before counting toward payment.
- Provider cannot confirm their own sessions.

9.7 Database Operations

Operation	Entity	Details
-----	-----	-----
Read	<code>ServiceExchange</code>	Status, credits, <code>completed_hours</code>
Read	<code>WorkSession</code>	Pending hours sum; last session number
Create	<code>WorkSession</code>	New pending session
Update (on confirm)	<code>ServiceExchange</code>	Increment <code>completed_hours</code> ; possibly COMPLETED
Update (on confirm)	<code>WorkSession</code>	Status $\rightarrow$ CONFIRMED

9.8 Security Considerations

- Provider-only recording enforced in service layer.
- Serializable transactions prevent concurrent over-recording.
- Participant checks on all exchange endpoints.

9.9 Error Handling

Condition	HTTP Status	Message
-----	-----	-----
Not provider	403	"Only the provider can record a session"
Inactive contract	400	"Contract is not active"
Hours exceed cap	400	"Total recorded hours cannot exceed agreed time credits"

9.10 Code References

Snippet 1 — Session recording with hours cap

File: `src/modules/exchanges/exchanges.service.ts`

```
export const recordWorkSession = async (contractId: number, providerId: number, data:
CreateSessionInput) => {
  const result = await runSerializable(async (tx) => {
    const exchange = await tx.serviceExchange.findUnique({ where: { id: contractId },
    ... });
    if (exchange.provider_id !== providerId) {
      throw new ExchangeError("Only the provider can record a session", 403);
    }
    const pendingHours = pendingSessions.reduce((acc, s) => acc + s.hours, 0);
    if (exchange.completed_hours + pendingHours + data.hours > exchange.time_credits) {
      throw new ExchangeError("Total recorded hours cannot exceed agreed time credits",
400);
    }
    const session = await tx.workSession.create({
      data: { contract_id: contractId, session_number: nextSessionNumber, hours:
data.hours, status: "PENDING_CONFIRMATION" },
    });
    return session;
  });
};
```

Snippet 2 — Session schema

File: `src/modules/exchanges/exchanges.schema.ts`

```
export const createSessionSchema = z.object({
  hours: z.coerce.number().int().positive().max(100000),
  notes: z.string().optional(),
});
```

Snippet 3 — WorkSession model

File: prisma/schema.prisma

```
model WorkSession {
  id          Int          @id @default(autoincrement())
  contract_id Int
  session_number Int
  hours       Int
  notes       String?
  status      WorkSessionStatus @default(PENDING_CONFIRMATION)
  @@unique([contract_id, session_number])
  @@map("work_sessions")
}
```

Snippet 4 — Session route

File: src/modules/exchanges/exchanges.routes.ts

```
router.post("/:id/sessions", validate(createSessionSchema), recordSessionController);
```

10. Resolve Contract at Maximum End Date

10.1 Purpose

This use case automatically terminates active service contracts that have reached their agreed `maximum_end_date` without manual intervention. The system pays the provider for confirmed work hours, refunds unused credits to the consumer, marks the contract as completed, and notifies both parties. Resolution is executed by a scheduled cron job, not by user request.

10.2 Actors

Actor	Role
-----	-----
Cron Scheduler	<code>node-cron</code> job in <code>contract-resolution.cron.ts</code>
Exchanges Module	<code>resolveExpiredContracts</code>
Provider	Receives earned credits
Consumer	Receives refund of unspent credits
Notifications Module	<code>CONTRACT_AUTO_RESOLVED</code> to both parties
PostgreSQL	<code>ServiceExchange</code> , <code>User</code> , <code>Transaction</code>

10.3 Preconditions

- Contract status is `IN_PROGRESS` or `WAITING_CONFIRMATION` .
- `maximum_end_date` <= current timestamp .
- Cron job is initialized at application startup (`startCronJobs()` in `server.ts`).

10.4 Trigger

Automatic — cron schedule `0 * * * *` (every hour at minute 0):

```
resolveExpiredContracts()
```

No HTTP endpoint exposes this operation.

10.5 Implementation Flow

1. 1. **Discovery** — Query all contracts where `status ∈ {IN_PROGRESS, WAITING_CONFIRMATION}` and `maximum_end_date ≤ now`.
2. 2. **Per-contract processing** — For each expired contract:
 - a. Begin serializable transaction.
 - b. Re-fetch contract with lock; skip if state changed or deadline no longer passed (optimistic re-check).
 - c. Calculate `providerCredits = completed_hours` and `refundCredits = time_credits - completed_hours`.
 - d. Transfer `providerCredits` to provider's `available_balance`; create `TRANSFER` transaction.
 - e. Refund `refundCredits` to consumer's `available_balance`; create `REFUND` transaction.
 - f. Update contract: `status: COMPLETED`, `escrow_status: RELEASED`, `completed_at: now`.
3. 3. **Notifications** — Both parties receive `CONTRACT_AUTO_RESOLVED`.
4. 4. **Error isolation** — Per-contract try/catch; one failure does not abort batch processing.

Related feature: providers may propose deadline extensions via `POST /exchanges/:id/deadline`, requiring consumer approval before `maximum_end_date` is updated.

10.6 Business Rules

- Provider receives only **confirmed** hours (`completed_hours`), not pending sessions.
- Unconfirmed/pending session hours are not paid out automatically.
- Remaining escrow (`time_credits - completed_hours`) is refunded to consumer.
- Contracts already completed, canceled, or rejected are excluded.
- Re-fetch inside transaction prevents resolving a contract that was just manually completed.

10.7 Database Operations

Operation	Entity	Details
-----	-----	-----
Read	<code>ServiceExchange</code>	Find expired active contracts
Update	<code>User</code> (provider)	Increment <code>available_balance</code>
Update	<code>User</code> (consumer)	Increment <code>available_balance</code> (refund)
Create	<code>Transaction</code>	<code>TRANSFER</code> and <code>REFUND</code> ledger entries
Update	<code>ServiceExchange</code>	<code>COMPLETED</code> , <code>RELEASED</code> , <code>completed_at</code>

10.8 Security Considerations

- No user-facing endpoint — only internal cron invocation.
- Serializable transactions prevent double-resolution.
- Re-check inside transaction handles concurrent manual completion.
- Escrow release is atomic with status update.

10.9 Error Handling

- Individual contract failures are logged via `console.error` and skipped.
- Cron wrapper catches top-level exceptions without crashing the server.
- Returns count of successfully resolved contracts for logging.

10.10 Code References

Snippet 1 — Cron scheduler

File: `src/common/cron/contract-resolution.cron.ts`

```
export const startCronJobs = () => {
  cron.schedule("0 * * * *", async () => {
    const resolvedCount = await resolveExpiredContracts();
    console.log(`[Cron] Successfully resolved ${resolvedCount} expired contracts.`);
  });
};
```

Snippet 2 — Expired contract discovery

File: `src/modules/exchanges/exchanges.service.ts`

```
const expiredContracts = await prisma.serviceExchange.findMany({
  where: {
    status: { in: ["IN_PROGRESS", "WAITING_CONFIRMATION"] },
    maximum_end_date: { lte: now },
  },
});
```

Snippet 3 — Payment and refund logic

File: `src/modules/exchanges/exchanges.service.ts`

```
const providerCredits = currentContract.completed_hours;
const refundCredits = currentContract.time_credits - providerCredits;

if (providerCredits > 0) {
  await tx.user.update({
    where: { id: currentContract.provider_id },
    data: { available_balance: { increment: providerCredits } },
  });
}
if (refundCredits > 0) {
  await tx.user.update({
    where: { id: currentContract.consumer_id },
    data: { available_balance: { increment: refundCredits } },
  });
}
await tx.serviceExchange.update({
  where: { id: currentContract.id },
  data: { status: "COMPLETED", escrow_status: "RELEASED", completed_at: now },
});
```

Snippet 4 — Cron initialization at startup

*File: `src/server.ts`

```
const app = express();
startCronJobs();
```

11. View Recommended Posts

11.1 Purpose

View Recommended Posts delivers a personalized post feed ranked by the FastAPI AI recommender using Sentence Transformers embeddings and FAISS similarity search. The backend acts as a **feed proxy**: it requests ordered post IDs from the AI service, hydrates full post objects from PostgreSQL preserving rank order, and falls back to chronological feed when the recommender is unavailable.

11.2 Actors

Actor	Role
-----	-----
Authenticated User	Consumes personalized feed on frontend
Recommender Module	<code>feedController</code> , <code>recommender.client.ts</code>
AI Service (FastAPI)	<code>POST /recommend</code> — returns ranked post IDs
Posts Module	<code>hydratePublishedPostsById</code> , shared pagination utilities
PostgreSQL	Published posts hydration

11.3 Preconditions

- Valid JWT access token.
- Recommender optionally configured via `RECOMMENDER_URL` and `RECOMMENDER_API_KEY` environment variables.
- User should exist in recommender index (synced at registration and on interactions).

11.4 Trigger

```
GET /feed/:userId?cursor=<postId>&limit=20
Authorization: Bearer <accessToken>
```

11.5 Implementation Flow

1. **Authentication** — `authMiddleware` on `feedRouter` .

2. 2. **Parameter validation** — `userId` parsed as positive integer; query params via `listPostsQuerySchema`.
3. 3. **AI recommendation request** — `fetchRecommendedPostIds(userId, 200)` calls `POST {RECOMMENDER_URL}/recommend` with `{ user_id, top_k: 200 }` and `X-Internal-Token` header. Timeout defaults to 5000 ms.
4. 4. **Cursor-based paging over AI order** — Unlike chronological feed, cursor is a **post ID** in the recommender's ordered list. System finds cursor index, slices next `limit + 1` IDs.
5. 5. **Hydration** — `hydratePublishedPostsById` loads posts from DB where `status: PUBLISHED`, preserving AI order (filters out unpublished/deleted).
6. 6. **Fallback** — If recommender disabled, times out, errors, or returns empty: `chronologicalFeed` executes same query as home feed. Response includes `source: "recommender" | "fallback"`.
7. 7. **Response** — `{ posts, nextCursor, source }`.

Integration architecture: **Backend-as-proxy** decouples the frontend from the Python AI service; the Express layer owns authentication, hydration, and graceful degradation.

11.6 Business Rules

- Only published posts appear in hydrated results.
- AI ordering preserved via ordered ID hydration (Map-based reorder).
- Maximum fetch cap: 200 IDs per recommender call (`RECOMMENDER_PAGE_FETCH_CAP`).
- Fallback is identical to `GET /posts` chronological logic.
- Interaction signals (apply, save) continuously update recommender via `syncInteraction`.

11.7 Database Operations

Operation	Entity	Details
-----	-----	-----
Read	<code>Post</code>	Hydration by ID list, filtered to PUBLISHED
Read	<code>User</code>	Nested author in <code>postSelect</code>

No writes during feed retrieval. Recommender index updates happen asynchronously on user actions elsewhere.

11.8 Security Considerations

- JWT required for feed endpoint.
- AI service protected by shared internal token (`X-Internal-Token`), not exposed to frontend.

- **Note:** Path param `:userId` is not validated against JWT subject — any authenticated user can request recommendations for any user ID (potential authorization gap).
- Recommender timeout prevents hung requests from blocking the feed.

11.9 Error Handling

Condition	HTTP Status	Behavior
-----	-----	-----
Invalid userId	400	"Invalid user id"
Invalid query	400	"Invalid request data"
Recommender unavailable	200	Fallback feed, <code>source: "fallback"</code>
Empty AI results	200	Fallback feed
Unexpected error	500	"Feed failed"

11.10 Code References

Snippet 1 — Feed controller with fallback

File: `src/modules/recommender/recommender.controller.ts`

```
export const feedController = async (req: Request, res: Response) => {
  const page = await recommendedFeed(userId, limit, query.cursor);
  if (!page) {
    return res.json(feedResponse(await chronologicalFeed(limit, query.cursor),
    "fallback"));
  }
  return res.json(feedResponse(page, "recommender"));
};
```

Snippet 2 — AI recommender client

File: `src/modules/recommender/recommender.client.ts`

```
export const fetchRecommendedPostIds = async (userId: number | string, topK = 20) => {
  const data = await request<RecommendResponse>("POST", "/recommend", {
    user_id: String(userId),
    top_k: topK,
  });
  return data.recommendations
    .map((item) => Number(item.post_id))
    .filter((id) => Number.isInteger(id));
};
```

Snippet 3 — Order-preserving hydration

File: src/modules/posts/posts.hydration.ts

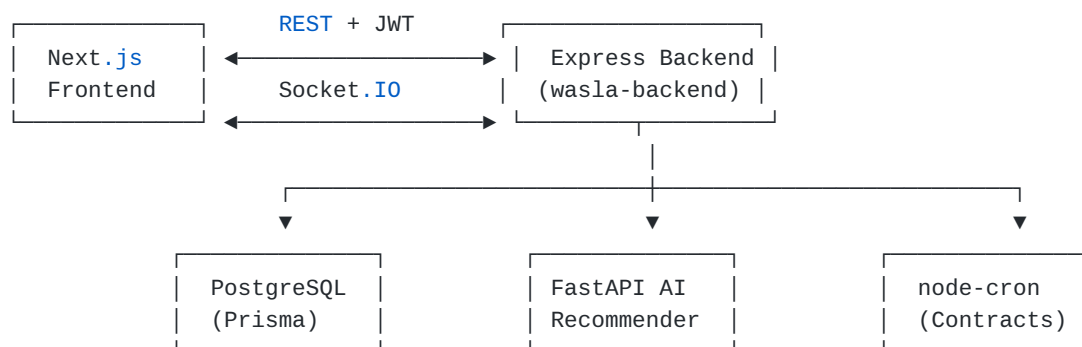
```
export const hydratePublishedPostsById = async (order: number[]) => {
  const posts = await prisma.post.findMany({
    where: { id: { in: order }, status: "PUBLISHED" },
    select: postSelect,
  });
  const byId = new Map(posts.map((post) => [post.id, post]));
  return order.map((id) => byId.get(id)).filter(Boolean).map(toPostResponse);
};
```

Snippet 4 — Feed route registration

File: src/modules/recommender/recommender.routes.ts

```
export const feedRouter = Router();
feedRouter.get("/:userId", authMiddleware, feedController);
```

Appendix A — System Architecture Summary



Key architectural patterns employed:

- **Modular monolith** — Feature modules (`auth` , `posts` , `exchanges` , `chat` , `recommender`) with routes → controllers → services → Prisma.
- **Middleware pipeline** — Authentication, validation (Zod), rate limiting applied declaratively on routes.
- **Escrow-based time credits** — Financial logic embedded in exchange service with serializable transactions.
- **AI proxy with graceful degradation** — Backend mediates recommender calls with chronological fallback.
- **Eventual consistency for AI index** — Fire-and-forget sync on registration, post creation, and interactions.
- **Real-time messaging** — Socket.IO for chat; polling-based notifications for contract events.

Document generated from analysis of the Wasla backend source code repository. All endpoints, schemas, and business rules reflect the implementation as of the current codebase state.